COMP2040 – Operating Systems

Group Project: ossim, a Kernel Simulator

Overview

In the labs each OS mechanism is a separate program. In a real kernel they run together: a process that page-faults leaves the CPU and waits for the disk, the end of the disk request puts it back in the ready queue, and a process blocked on a semaphore is invisible to the scheduler until someone signals. In this project you complete `ossim`, a simulator in C where all of this happens at the same time, and then use it for experiments.

You work in groups of 3–4 and start from `ossim.zip` on Canvas. It already runs with the simplest policy everywhere. The project has two parts:

- **Core**, the same for every group: the remaining schedulers, deadlock detection, page replacement, disk scheduling, a workload generator, tests and a report (Sections 3–5).
- **Theme**: one extension chosen by your group, studied with one experiment (Section 6).

Checkpoint 1: progress report 1 and code	Week 8
Checkpoint 2: progress report 2 and code, theme choice	Week 12
Last week to change your theme	Week 13
Final report, code and presentation	Week 15

1 How ossim works

A workload file lists processes. Each process is a sequence of operations: `cpu N` (run N ticks, one page reference per tick), `io N`, `disk C` (a request on cylinder C), `wait S` and `signal S` on semaphores, and `acquire R n` and `release R n` on resources. The format is in `include/workload.h`; the samples are in `workloads/`.

```
make
make test
./ossim workloads/basic.txt --sched rr --quantum 4 --no-paging --gantt
./ossim workloads/paging_p1.txt --frames 3 --replace fifo --trace
```

Time moves one tick at a time. In every tick, `src/kernel.c` does this:

1. Deliver the events due now: arrivals, end of io, end of a disk request.
2. Ask the scheduler whether to preempt the running process. If the CPU is free, dispatch the process the scheduler picks.
3. Count the tick as waiting for every READY process and as blocked for every WAITING one.
4. Run one tick of the running process. It references one page. On a hit one tick of the burst is done. On a page fault the tick goes to the kernel, the process blocks and a page-in is sent to the disk.

At the end the report gives, per process, arrival, first run, finish, turnaround, waiting and response time, and a summary with CPU utilisation, context switches, page faults, head movement and more, as `key=value` lines.

Rules of the simulation

You need these to compute results by hand.

- A burst of N ticks that starts at tick t finishes at $t + N$. Waiting time is the number of ticks spent READY. Response time is the first dispatch minus the arrival, so it includes

time blocked before the first burst.

- Events at the same tick are handled in the order they were created. Arrivals are created first, so an arriving process enters the ready queue before one that comes back from io or is preempted in that tick.
- `signal`, `release`, and a `wait` or `acquire` that does not block take no time. A process whose next operation is `cpu` again keeps the CPU; the scheduler may still preempt it in the next tick.
- RR: when the quantum ends and nobody else is ready, the process keeps the CPU and starts a new quantum. A context switch is counted each time a different process is dispatched, the first dispatch included.
- Disk: 200 cylinders, the head starts at `--head` (default 0) moving `--direction` (default up). A request takes $\lceil \text{cylinders moved}/10 \rceil + 1$ ticks by default. A page-in goes to cylinder $(pid \cdot 31 + page \cdot 17) \bmod 200$. If every frame is still loading a page, the faulting process waits for a frame and tries again.

2 Adding a policy

A scheduler is a struct of functions (`include/sched.h`) that the kernel calls at fixed points: `enqueue` when a process becomes READY, `should_preempt` before the running process runs a tick, `pick` when the CPU is free, `on_dispatch` and `on_tick`. FCFS and RR in `src/sched.c` are complete examples. A scheduler that always runs the lowest pid looks like this:

```
static queue_t pid_q;
static int  pid_key(const pcb_t *p)      { return p->pid; }
static void pid_init(void)               { q_init(&pid_q); }
static void pid_enqueue(pcb_t *p)        { q_push(&pid_q, p); }
static bool pid_should_preempt(pcb_t *r) { (void)r; return false; }
static pcb_t *pid_pick(void)             { return q_pop_min(&pid_q, pid_key); }
static void pid_on_dispatch(pcb_t *p)    { (void)p; }
static void pid_on_tick(pcb_t *r)        { (void)r; }
static int  pid_ready_count(void)        { return q_size(&pid_q); }

static scheduler_t sched_pid = { "pid", pid_init, pid_enqueue,
    pid_should_preempt, pid_pick, pid_on_dispatch, pid_on_tick, pid_ready_count };
```

Add `&sched_pid` to the `all[]` array at the bottom of the file and run it with `--sched pid`. The values in the struct must be in the same order as the fields of `scheduler_t`. `(void)r;` is needed because `-Werror` stops the build on an unused parameter. Replacement policies (`src/mem.c`) and disk policies (`src/disk.c`) work the same way.

For every policy, work out a small case by hand and add a check to `tests/run.sh`:

```
check "SJF on arrivals.txt" \
      "$BIN workloads/arrivals.txt --sched sjf --no-paging" "avg_waiting=..."
```

`make test` prints SKIP for a policy that is not written yet, and PASS or FAIL once it is. Test schedulers with `--no-paging`, so page faults do not change the timing.

3 Checkpoint 1 (Week 8): scheduling and deadlock

(a) In `src/sched.c`: SJF (non-preemptive), SRTF, preemptive priority (smaller number is higher priority, RR between equal priorities), and MLFQ with three levels and quanta q, $2q$, $4q$. In MLFQ a process that uses its whole quantum moves down one level, one that blocks earlier stays, and every 20 ticks all processes go back to level 0.

(b) In `src/sync.c`: `deadlock_check()`, with the wait-for graph or the detection algorithm of textbook Section 8.6.2. It prints `DEADLOCK t=<tick> pids: 1 2` (pids in increasing order) and must find A and B in `workloads/deadlock.txt`. Recovery is optional: to kill a victim, call `kernel_abort(p)`.

(c) `workloads/producer_consumer.txt` must finish under every scheduler.
(d) A check in `tests/run.sh` for each scheduler, with values you computed by hand. `workloads/arrivals.txt` is a good case.

Submit the code with your tests, and progress report 1 of at most two pages: the data structures of your schedulers, how you checked them, and who did what.

4 Checkpoint 2 (Week 12): memory, disk and the generator

(a) In `src/mem.c`: LRU (with `last_used`) and Clock (second chance, with `referenced`). With 3 frames, `workloads/paging_p1.txt` gives 15 faults under FIFO and 12 under LRU. Make your own hand-computed case for Clock.
(b) In `src/disk.c`: SSTF, SCAN and C-LOOK. With `workloads/disk_textbook.txt` and `--head 53`, the head movement is 640 (FCFS), 236 (SSTF), 236 (SCAN, `--direction down`) and 322 (C-LOOK, `--direction up`). SCAN reports its trip to the edge of the disk through `via[]`, see `include/disk.h`.
(c) A workload generator, in any language, with options for the number of processes, burst lengths, I/O, locality of page references and disk request patterns.

Submit the code, tests, generator, and progress report 2: the new parts, how you checked them, who did what, and the theme you chose (Section 6) with a few lines on your plan.

5 Final submission (Week 15)

Submit the complete simulator, the generator, the tests and a report of 8–10 pages with: the design, the checks against hand-computed cases, and at least three experiments, one of them on your theme. Each experiment has a question, a method, a plot or table, and a discussion of the result. Some questions for the core:

- How do waiting and response time of the six schedulers change with the amount of I/O?
- How does the quantum affect context switches, response time and page faults?
- Where does thrashing start as the number of frames goes down, for FIFO, LRU and Clock? Does Belady's anomaly appear for FIFO but not for LRU?
- How do the disk schedulers compare in head movement and waiting time for random and for local requests, and how does that change page-fault service time (`avg_fault_service`)?

Each group gives a 10-minute presentation with a demo of the simulator, followed by questions to each member. Any member may be asked about any part of the simulator. Each member also submits a confidential peer assessment.

6 Themes

Choose one by Checkpoint 2. Several groups may choose the same theme, but code is not shared between groups. The extension is turned on by a command-line option, so a run without it behaves as before. Include at least one check you computed by hand, one experiment in the report, and show it in the presentation. A theme of your own of the same size is fine if you agree it with us before Week 12.

1. **Working set and load control.** Track each process's working set over a window Δ and suspend a process when the working sets do not fit in memory. Check: the textbook example with $\Delta = 10$. Does it stop thrashing?
2. **Local and global allocation.** Equal and proportional allocation, where a fault may only evict the process's own pages. Check: 62 frames for processes of 10 and 127 pages give 4 and 57. Who pays for a process with poor locality?
3. **Priority inheritance.** A lock remembers its holder, and the holder gets the priority of

the highest process waiting for it. Check: the three-process inversion example as a Gantt chart. How long does the high-priority process wait, with and without it?

4. **Lottery or stride scheduling.** Tickets per process in the workload. Check: a stride schedule for tickets 3:2:1 by hand. How fast does the CPU share reach the ticket share?
5. **Dirty pages.** Writes in the reference string, a dirty bit, and a write-back to disk before a dirty page is replaced. Check: a short string with writes under FIFO. What does the write ratio do to page-fault service time?
6. **Two disks.** RAID-0: two disks, each with its own head, queue and scheduler, and a stripe mapping of your choice. Check: the textbook queue split by your mapping. How much faster is page-fault service under memory pressure?
7. **Banker's algorithm.** Maximum claims in the workload and the safety check in `res_acquire`. Check: the textbook example with resources (10, 5, 7). How much throughput does avoidance cost compared with detection?
8. **Deadlock recovery.** Kill a victim with `kernel_abort()`, or restart it, with at least two ways to choose the victim. Check: the detection example in textbook Section 8.6.2. How does the detection interval trade unnoticed deadlock against lost work?

7 Rules

- The code must build with `make` using `gcc` or `clang` with `-std=c99 -Wall -Wextra -Werror`, on Linux, macOS or WSL, with no other libraries.
- Use one git repository per group, with regular commits from every member. The history is part of the evidence of who did what.
- You may use AI tools and online material to understand and to debug, but every line you submit must be understood.

8 Marks

Part	Weight	What is marked
Checkpoint 1	10%	progress report 1 and code (Section 3)
Checkpoint 2	20%	progress report 2 and code (Section 4)
Final	70%	report, code, presentation and theme (Sections 5 and 6)